Vibe Coding 实操示例（课堂学情监控系统场景）

前置说明

- **使用工具**：Cursor / Trae 这类AI原生代码编辑器（AI可直接读写本地项目文件，无需复制粘贴）
- **技术栈**：后端 SpringBoot + MySQL，前端 HTML+CSS+原生JavaScript
- **初始状态**：只有一个空项目文件夹，没有提前写完整的需求文档、数据库设计，只知道「要做一个课堂学情反馈系统」的大致目标
- **核心逻辑**：想到哪说到哪，每次只说一件事；AI改完立刻运行验证，不对就接着改，小步迭代逐步完善

完整迭代过程模拟

第1步：开局搭骨架（先让项目跑起来）

你对AI说的提示词

> 帮我搭一个 SpringBoot + MySQL 的 Web 项目骨架，做课堂学情监控系统用，能直接启动就行，先不用写业务逻辑。

AI完成的动作

- 生成 pom.xml 依赖、项目启动类、application.yml 数据库配置
- 创建基础的包结构（controller、service、entity）
- 生成数据库连接配置，自带启动测试接口

你要做的验证

- 点击启动项目，看控制台是否正常启动、端口是否正常
- 测试数据库连接是否成功，不报错就进入下一步

> 特点：开局不纠结细节，先拿到一个能跑的空架子。

第2步：加第一个核心功能：学生表情提交

你对AI说的提示词

> 加一个学生端页面 feedback.html，页面上放6个表情按钮，分别对应1-6分，6分是完全理解，1分是完全没理解。点击按钮提交到 POST /api/feedback，参数传 studentId 和 score，提交成功弹个提示框。

AI完成的动作

- 生成前端 feedback.html 页面，6个按钮带点击事件
- 写好 FeedbackController、Service、实体类
- 实现提交数据插入 feedback 表的逻辑
- 加提交成功/失败的前端提示

你要做的验证

- 浏览器打开页面，随便点一个表情
- 去数据库看有没有新增一条反馈记录
- 能正常提交就进入下一步，样式丑、逻辑不完美都没关系

第3步：加教师端统计展示

你对AI说的提示词

> 再加一个教师端页面 stats.html，显示当前提交的学生人数、平均分，再用柱状图画出来每个分数分别有多少人。

AI完成的动作

- 生成教师统计页面，加入数字展示区域
- 写统计查询接口，计算提交人数、各分数段人数、平均分
- 引入 ECharts 生成柱状图

你要做的验证

- 用几个学生账号提交不同分数
- 刷新教师页，看数字和图表对不对得上

第4步：发现问题，当场修正（vibe coding 核心：不对就改）

你验证的时候发现：同一个学生提交两次，人数居然算成了2个，逻辑不对。

你对AI说的提示词

> 统计逻辑改一下：每个学生只算最后一次提交的分数，不要把所有提交记录都算进去，提交人数也按去重后的学生数算。

AI完成的动作

- 修改查询逻辑，按学生ID分组取最新提交时间
- 重新计算人数、各分数段人数、平均分

你要做的验证

- 同一个学生连续提交两次不同分数
- 确认人数不增加、分数以最后一次为准

> 特点：不用提前想好所有规则，遇到问题当场提、当场改。

第5步：继续加功能：定时统计

你对AI说的提示词

> 加一个定时任务，每5分钟自动统计一次全班学情，结果存到单独的统计表里，历史数据要能保留下来。

AI完成的动作

- 新建学情统计表
- 加 @Scheduled 定时任务，每5分钟执行一次统计计算
- 把统计结果持久化到统计表里

你要做的验证

- 把定时改成1分钟测试，看到点会不会自动生成记录
- 确认历史记录不会被覆盖

第6步：继续补功能：历史查询 + 导出Excel

你对AI说的提示词

> 教师页面加个历史查询区域，可以按课程名称和时间范围查历史统计；再加个导出按钮，把查询结果导出成 Excel。

AI完成的动作

- 页面加查询表单和查询按钮
- 新增历史查询接口、Excel导出接口
- 点击导出自动下载 .xlsx 文件

你要做的验证

- 输入时间范围查询，看结果对不对
- 点导出，看能不能正常下载Excel、字段齐不齐

第7步：补权限：登录 + 角色区分

你对AI说的提示词

> 加个登录页面，支持账号密码登录，区分学生、老师、管理员三个角色，学生只能进反馈页，老师只能进统计页，管理员可以管理用户。

AI完成的动作

- 新建用户表，加角色字段
- 写登录接口、会话拦截器
- 不同角色登录后跳转到对应页面
- 加简单的用户管理页面

第8步：收尾优化：调样式、修小bug

你对AI说的提示词

> 把学生页和教师页的样式调一下，适配手机端，按钮大一点，颜色区分开，当前时段的统计结果高亮显示。

AI完成的动作

- 重写CSS，加响应式布局
- 调整按钮大小、配色

- 当前时段数据加粗高亮

Vibe Coding 核心操作要点

1. **每次只说一件事** 不要一次把所有功能都说完，一次只提一个小功能/一个小问题，AI不容易混乱。
2. **改完立刻验证** 每改完一步就跑一下，不对马上接着说，小步快跑，不要攒一大堆再测。
3. **不用提前规划全部细节** 方差、饼图、短信登录这些细节，可以后面慢慢加，不用第一天就全想好。
4. **边做边对齐需求** 做到一半回头看一眼产品要求文档，漏掉的功能随时补，避免最后漏项扣分。

适合场景与提醒

- 适合：快速出Demo、小项目、课程大作业原型验证
- 不适合：大型项目、多人协作、需求非常严谨的正式项目
- 给学生的建议：先用 Vibe coding 快速做出可运行版本，最后再对照产品要求文档逐项核对补全，兼顾效率和不丢分。